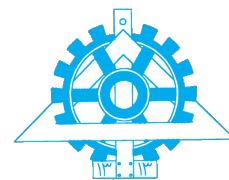




University of Tehran
College of Engineering
School of Electrical and Computer Engineering



Reinforcement Learning Final Project

Competitive Dynamic Pricing

Name

Fateme Roshani (810104151)
Amirhesam Kamalpour (810104235)

February 23, 2026

Abstract

Pricing is an important decision in competitive markets because it directly affects demand and long-term profit. In this project, we model competitive dynamic pricing as a sequential decision-making problem and solve it using reinforcement learning. A firm sets a continuous price for one product at discrete time steps, while a competitor also changes its price dynamically. Demand is stochastic and depends on both prices and a time-varying baseline that includes slow growth and seasonality. Because the agent does not directly observe the latent baseline demand, the problem is partially observable and the agent must infer dynamics from a history window of past prices and demand. We implement a custom simulator and train a Soft Actor-Critic (SAC) agent. The results show that the agent quickly improves after learning begins, reaches stable high returns, and learns a pricing strategy that stays slightly above the competitor while responding indirectly to demand seasonality.

1 Introduction

In many markets, firms cannot choose one fixed price and keep it forever, because demand changes over time and competitors react. In real settings like online retail or ride-sharing, companies repeatedly update prices to increase profit and keep their market position. However, finding an optimal pricing strategy is difficult because demand is uncertain, competitor actions are strategic, and the market changes over time. Classical optimization methods often require a strong demand model and can fail when the environment is dynamic.

Reinforcement learning is suitable for this problem because it learns a policy through interaction, without needing a perfect analytical model. The agent can try prices, observe outcomes, and gradually improve to maximize long-term profit. In this project, we focus on a single-product case and build a simulator that includes competition, stochastic demand, and time-dependent baseline demand. We then train a model-free RL agent with continuous actions.

2 Problem Formulation

2.1 Market Setting

In this project, we consider a competitive market where two firms sell a single product. One firm is the focal firm, which is controlled by a reinforcement learning (RL) agent. The other firm acts as a dynamic competitor whose price changes over time according to a predefined behavioral model. Both firms set prices repeatedly at discrete time steps, and market demand is influenced by the relationship between these prices.

The objective of the RL agent is to choose prices that maximize long-term profit while operating in a stochastic and competitive environment. The agent does not directly control the competitor, but it must learn how competitor behavior affects demand and profitability. This setup captures the strategic nature of pricing decisions in real markets, where firms react to each other instead of acting independently.

For simplicity, the problem assumes a single-product setting and ignores inventory limitations or cross-product effects. Prices are continuous and bounded within a fixed range, which makes the control problem more realistic compared to discrete pricing models.

2.2 MDP / POMDP Definition

The pricing problem is formulated as a sequential decision-making task. At each time step t , the agent selects a price action

$$p_t \in [p_{\min}, p_{\max}], \quad (1)$$

where $p_{\min}$ and $p_{\max}$ define the allowed pricing range.

After the action is chosen, the competitor sets its own price, demand is realized, and a reward is received. The reward represents the firm’s profit with an additional regularization term that discourages large price changes:

$$r_t = (p_t - c) q_t - \lambda |p_t - p_{t-1}|, \quad (2)$$

where c is the unit cost, q_t is demand, and λ controls price smoothness. This design encourages the agent to learn stable pricing behavior instead of frequently changing prices.

The environment evolves over a finite horizon, meaning each episode contains a fixed number of decision steps. The agent aims to maximize the accumulated discounted reward across the episode.

An important characteristic of this problem is that the agent does not observe the full market state. Hidden factors such as latent demand trends and seasonality affect demand but are not directly visible. To deal with this limitation, the observation given to the agent consists of a history window containing past prices and demand values:

- past agent prices,
- past competitor prices,
- past demand observations.

This approach allows the agent to infer hidden dynamics from previous information. Therefore, the problem is more accurately described as a Partially Observable Markov Decision Process (POMDP). In practice, partial observability is handled by stacking historical observations, which provides the agent with enough temporal context to approximate the underlying market state.

3 Environment Simulator Design

To train and evaluate the reinforcement learning agent, we built a simulated competitive market environment. The simulator is designed to be simple enough to train efficiently, but realistic enough to include important features of real dynamic pricing problems, such as competition, stochastic demand, and time-varying market conditions. In each episode, the environment runs for a fixed horizon H , and at each discrete step t , both firms set prices and the market demand is realized.

3.1 Decision Timeline

At every time step, the environment follows a fixed order of events. First, the focal firm (the RL agent) selects a price p_t . At the same time, the competitor sets its price p_t^c . Even though the pricing is “simultaneous,” the competitor’s decision depends only on past values, so the environment transition remains well-defined. After both prices are set, demand q_t is generated based on the prices and the baseline market demand. Finally, the reward r_t is calculated and the history-based state is updated for the next step.

3.2 Competitor Pricing Dynamics

The competitor is modeled using an autoregressive reaction function with noise. In the general form, the competitor price follows an ARX model with lag length m :

$$p_t^c = a + \sum_{i=1}^m b_i p_{t-i} + \sum_{i=1}^m d_i p_{t-i}^c + u_t, \quad u_t \sim \mathcal{N}(0, \sigma_{\text{comp}}^2). \quad (3)$$

In our implementation, we used $m = 1$, which means the competitor reacts to the most recent prices:

$$p_t^c = a + b p_{t-1} + d p_{t-1}^c + u_t. \quad (4)$$

This model captures two realistic behaviors. First, the coefficient b represents how the competitor reacts to the focal firm’s previous price. Second, the coefficient d captures inertia, meaning the competitor tends to keep its own price near its past value. To avoid unrealistic values, the competitor price is clipped to remain within the allowed price range.

3.3 Demand Model

Demand is modeled as a noisy response to both the focal firm’s own price and the price difference with the competitor. At time t , demand is computed as:

$$q_t = \max\left(0, \alpha_t - \beta p_t + \eta(p_t^c - p_t) + \epsilon_t\right), \quad \epsilon_t \sim \mathcal{N}(0, \sigma_{\text{demand}}^2). \quad (5)$$

Here, $\beta > 0$ controls how strongly demand decreases when the agent raises its own price. The term $\eta(p_t^c - p_t)$ represents a competitive substitution effect: if p_t^c becomes higher than p_t , this term increases demand for the agent, matching the idea that customers may switch to the cheaper seller. The $\max(0, \cdot)$ function ensures demand is non-negative.

3.4 Time-Varying Baseline Demand (α_t)

To make the environment dynamic, baseline demand α_t changes over time using a combination of log-growth and seasonality. We define a numerically stable growth term:

$$\ell(t) = \log\left(1 + \frac{t}{T}\right), \quad (6)$$

where T is the seasonal period. Then baseline demand is:

$$\alpha_t = (\alpha_0 + g \ell(t)) \left(1 + s \sin \left(2\pi \frac{t}{T} + \phi \right) \right). \quad (7)$$

In this equation, α_0 is the initial baseline demand, g controls the strength of long-term growth, s controls seasonal amplitude, and ϕ is a phase shift. This design creates a market where demand generally increases early on but the growth slows down over time because of the logarithmic term. At the same time, the sine component creates periodic seasonal fluctuations.

In our setting, α_t is treated as a hidden driver unless time features are directly included in the observation. Since we excluded explicit time features, the agent must infer these patterns indirectly through demand history.

3.5 Reward Function and Cost Assumptions

The reward is designed to represent the firm’s profit while discouraging unstable pricing. The profit at time t is:

$$\text{profit}_t = (p_t - c) q_t, \quad (8)$$

where c is the constant unit production cost. We then add a smoothness penalty that depends on the absolute price change:

$$r_t = (p_t - c) q_t - \lambda |p_t - p_{t-1}|. \quad (9)$$

The parameter λ controls how strongly the agent is penalized for frequent price changes. This term is similar to menu costs in economics, where changing prices can have real costs, and it helps the agent learn more stable pricing behavior instead of oscillating prices.

3.6 Observation Space and Partial Observability

The agent does not observe the full state of the market, so we use a history-based observation. At each time step, the agent receives a rolling window of length k containing:

$$\tilde{s}_t = [p_{t-k:t-1}, p_{t-k:t-1}^c, q_{t-k:t-1}]. \quad (10)$$

This observation includes the agent’s recent prices, the competitor’s recent prices, and recent demand values. Since time features such as $\sin(2\pi t/T)$ and $\cos(2\pi t/T)$ are not included, the agent must learn to estimate hidden seasonality and growth effects from historical signals. As a result, the environment is closer to a POMDP than a fully observable MDP, but history stacking provides the agent with enough temporal information to learn a useful policy in practice.

4 Reinforcement Learning Methodology

In this project, the pricing problem is solved using a model-free deep reinforcement learning method. Since the agent must choose a continuous price value at each step, we need an

algorithm that can handle continuous action spaces and learn efficiently from experience. For this reason, we used **Soft Actor-Critic (SAC)**, which is an off-policy actor-critic algorithm designed for stable learning in continuous-control tasks.

4.1 Why Soft Actor-Critic (SAC)

Soft Actor-Critic is suitable for competitive dynamic pricing for three main reasons. First, the action space is continuous, because the agent chooses a real-valued price $p_t \in [p_{\min}, p_{\max}]$. SAC directly models a stochastic policy over continuous actions, which is more natural than discretizing the price space.

Second, SAC uses **entropy regularization** to encourage exploration. Instead of only maximizing expected return, SAC maximizes a trade-off between return and policy entropy. The standard SAC objective can be written as:

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{H-1} \gamma^t \left(r_t + \alpha \mathcal{H}(\pi(\cdot \mid o_t)) \right) \right], \quad (11)$$

where $\mathcal{H}(\pi(\cdot \mid o_t))$ is the entropy of the policy at observation o_t , and α is the temperature parameter that controls how much exploration is encouraged. This is important in our setting because the environment is stochastic and partially observable, so exploration helps the agent discover profitable price regions and robust strategies.

Third, SAC is an **off-policy** algorithm. This means it learns from a replay buffer that stores past transitions, which improves sample efficiency. Since our environment is simulated but still relatively complex, learning from stored experiences makes training more stable and faster.

4.2 Policy and Value Function Architecture

As in the figure 1 shows SAC uses two main components: a stochastic policy network (actor) and value networks (critics).

4.2.1 Actor: Squashed Gaussian Policy

The actor models a Gaussian distribution over actions and then applies a tanh function to keep outputs bounded. In the unscaled form, the actor outputs the mean $\mu(o)$ and standard deviation $\sigma(o)$ of a Gaussian:

$$z \sim \mathcal{N}(\mu(o), \sigma(o)^2), \quad u = \tanh(z), \quad (12)$$

where $u \in [-1, 1]$. Since the actual action must be in the environment’s price range $[p_{\min}, p_{\max}]$, the action is linearly scaled:

$$p = u \cdot \frac{p_{\max} - p_{\min}}{2} + \frac{p_{\max} + p_{\min}}{2}. \quad (13)$$

Because tanh and scaling change the probability density, SAC corrects the log-probability using a change-of-variables term. In our implementation, we compute the log-probability of the sampled action and apply the tanh-squash correction, and we also apply an additional correction for the linear scaling.

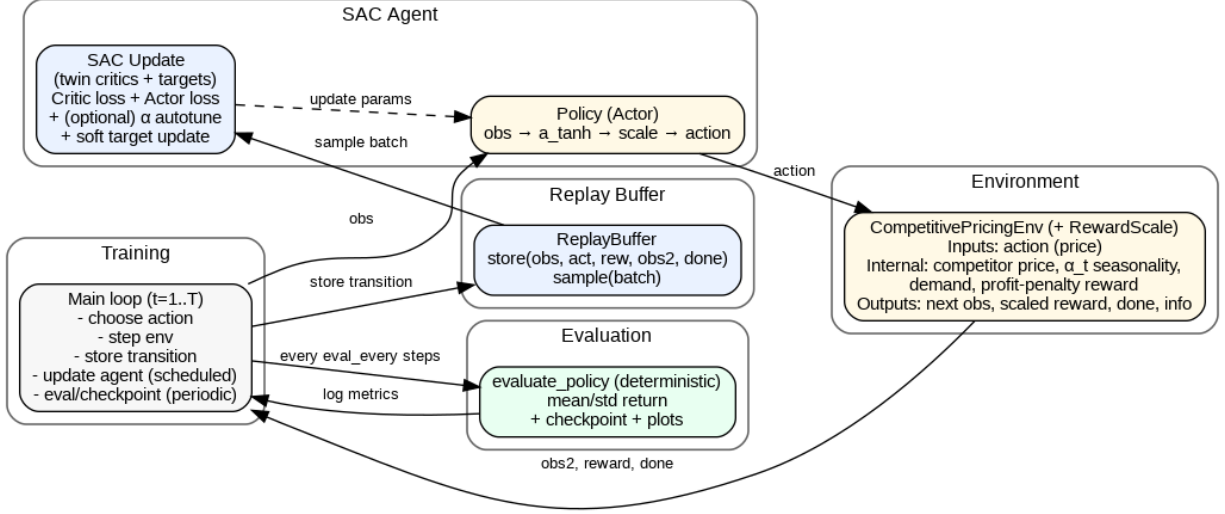


Figure 1: SAC Competitive Pricing Flow

4.2.2 Critics: Twin Q-Networks

SAC uses two Q-functions, $Q_{\theta_1}(o, a)$ and $Q_{\theta_2}(o, a)$, to reduce overestimation bias. The target value uses the minimum of the two critics:

$$Q_{\min}(o, a) = \min(Q_{\theta_1}(o, a), Q_{\theta_2}(o, a)). \quad (14)$$

This is important for stability, especially in noisy environments where value estimates can become optimistic.

4.3 SAC Update Rules and Objective Functions

The critics are trained to match a Bellman backup target. With entropy regularization, the target is:

$$y = r + \gamma(1 - d)(Q_{\min}(o', a') - \alpha \log \pi(a' | o')), \quad (15)$$

where d is the done flag, o' is the next observation, and $a' \sim \pi(\cdot | o')$. Each critic is trained using mean squared error:

$$L_Q(\theta_i) = \mathbb{E}[(Q_{\theta_i}(o, a) - y)^2], \quad i \in \{1, 2\}. \quad (16)$$

The policy is trained to maximize the expected Q-value while also keeping entropy high:

$$L_\pi(\phi) = \mathbb{E}[\alpha \log \pi_\phi(a | o) - Q_{\min}(o, a)], \quad a \sim \pi_\phi(\cdot | o). \quad (17)$$

In addition, SAC uses **soft target networks** for stability. Target critic parameters are updated slowly:

$$\theta_{\text{targ}} \leftarrow (1 - \tau)\theta_{\text{targ}} + \tau\theta, \quad (18)$$

where τ is a small constant (in our setup, $\tau = 0.005$).

4.4 Automatic Entropy Tuning

Instead of using a fixed α , we used automatic entropy tuning. The idea is to adjust α so that the policy entropy stays near a target value. A common choice is:

$$\mathcal{H}_{\text{target}} = -\dim(\mathcal{A}), \quad (19)$$

which in our case is -1 because the action is one-dimensional (price). SAC optimizes α by minimizing:

$$L_\alpha = -\mathbb{E}[\log \alpha (\log \pi(a \mid o) + \mathcal{H}_{\text{target}})]. \quad (20)$$

This helps the agent explore more at the beginning and become more deterministic later, which is useful for convergence.

4.5 Training Setup and Practical Details

Training was done in a simulated environment with the following setup:

- Total interaction steps: **1400**
- Random exploration steps (warm-up): **200**
- Updates start after: **100** steps
- Updates performed every step with **one gradient update per step**
- Evaluation every **20** steps using **5** evaluation episodes

We used a replay buffer to store transitions $(o_t, a_t, r_t, o_{t+1}, d_t)$, and mini-batches were sampled randomly for updates. This reduces correlation between samples and improves stability.

Finally, we applied **reward scaling** by multiplying the environment reward by a constant factor (0.01). This is a practical trick that helps prevent very large target values in the critic updates, which can make learning unstable. In our setting, reward scaling made training smoother and helped SAC learn a stable policy within a small number of steps.

5 Evaluation Protocol

To evaluate the performance of the reinforcement learning agent, we used a consistent testing procedure during training and after training. The main goal of evaluation was to measure whether the learned pricing policy improves long-term profitability while maintaining reasonable and stable pricing behavior in a competitive and stochastic market.

5.1 Periodic Evaluation During Training

Evaluation was performed periodically during the training loop. Specifically, after every fixed number of environment interaction steps (in our implementation, every 20 steps), the current policy was tested in a separate evaluation environment. During evaluation, the agent used a **deterministic policy**, meaning that it selected the mean action from the learned policy network instead of sampling a stochastic action. This choice reduces randomness and helps measure the true performance of the learned policy.

For each evaluation checkpoint, we ran a fixed number of evaluation episodes ($N = 5$ episodes). Each episode lasted for a full horizon H , and we computed the total return for that episode as the sum of rewards across time:

$$G = \sum_{t=0}^{H-1} r_t. \quad (21)$$

Because the environment is stochastic, a single episode can be noisy. Therefore, we reported both the mean and the standard deviation of evaluation returns across the evaluation episodes:

$$\mu_G = \frac{1}{N} \sum_{i=1}^N G_i, \quad \sigma_G = \sqrt{\frac{1}{N} \sum_{i=1}^N (G_i - \mu_G)^2}. \quad (22)$$

These two values help us understand not only the expected performance of the policy, but also how stable the performance is under randomness.

5.2 Key Metrics

Although the evaluation return is the main overall score, we also tracked additional metrics that describe the quality of the learned pricing behavior.

5.2.1 (1) Profit-based performance

The reward in the environment is based on profit with a smoothness penalty:

$$r_t = (p_t - c) q_t - \lambda |p_t - p_{t-1}|. \quad (23)$$

Therefore, a higher return G generally means the agent is achieving stronger profit over time while also controlling excessive price changes.

5.2.2 (2) Price stability

To measure stability more directly, we used the average absolute price change:

$$\text{Mean Price Change} = \frac{1}{H-1} \sum_{t=1}^{H-1} |p_t - p_{t-1}|. \quad (24)$$

We also considered the general variability of the price sequence through its variance:

$$\text{Var}(p) = \frac{1}{H} \sum_{t=0}^{H-1} (p_t - \bar{p})^2, \quad \bar{p} = \frac{1}{H} \sum_{t=0}^{H-1} p_t. \quad (25)$$

5.2.3 (3) Demand behavior and sanity checks

Because demand is generated as

$$q_t = \max\left(0, \alpha_t - \beta p_t + \eta(p_t^c - p_t) + \epsilon_t\right), \quad (26)$$

we also monitored demand patterns during rollouts. In particular, we tracked the frequency of zero-demand events:

$$\text{Zero-demand rate} = \frac{1}{H} \sum_{t=0}^{H-1} \mathbb{I}[q_t = 0], \quad (27)$$

where $\mathbb{I}[\cdot]$ is an indicator function. A high zero-demand rate can suggest unrealistic pricing behavior or that the agent is setting prices too high for the market.

5.3 Post-Training Diagnostic Rollout

After training finished, we performed one full rollout using the final deterministic policy and recorded step-level variables such as the agent price p_t , competitor price p_t^c , baseline demand α_t , realized demand q_t , and reward r_t . These logged trajectories were then plotted to visually inspect how the learned policy behaves over time, especially in relation to competitor behavior and the changing demand baseline. This diagnostic analysis is useful because it explains why a certain return was achieved and whether the policy’s behavior is reasonable in a competitive market setting.

6 Results

In this section, we report the main outcomes of training the SAC agent in the competitive pricing simulator. During training, we evaluated the deterministic policy every 20 environment steps and averaged the episode return over 5 evaluation episodes. The episode return is the total discounted-free sum over the horizon:

$$G = \sum_{t=0}^{H-1} r_t, \quad (28)$$

and for each evaluation checkpoint we report the mean and standard deviation:

$$\mu_G = \frac{1}{N} \sum_{i=1}^N G_i, \quad \sigma_G = \sqrt{\frac{1}{N} \sum_{i=1}^N (G_i - \mu_G)^2}, \quad (29)$$

where $N = 5$. (In the implementation, rewards were scaled by 0.01 before learning, so the plotted/evaluated returns are on the scaled reward.)

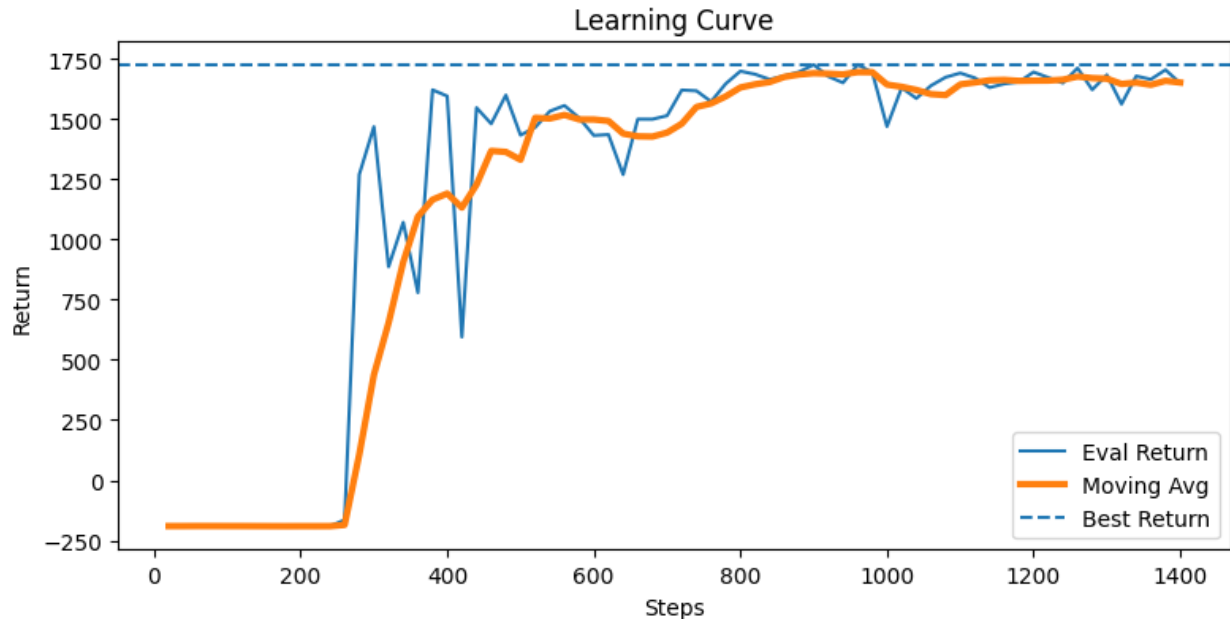


Figure 2: Learning Curve

6.1 Learning Curve and Performance Improvement

As in the figure 2 the learning curve shows three clear phases. First, for the early part of training (up to around 240 steps), the evaluation return stays almost constant near -190 . This happens because the policy network is not trained yet, and the deterministic actions produced by the initial actor lead to low profit. The printed logs confirm this, because until step 240 the evaluation return mean remains around -190 with small standard deviation.

Second, after learning updates start to take effect (around steps 260–300), the evaluation return increases sharply. For example, the mean return moves from approximately -165 at step 260 to above 1200 at step 280, and then reaches around 1467 at step 300. This indicates that the agent quickly learns a much better pricing region once the replay buffer has enough data and gradient updates begin.

Third, after this rapid improvement, performance becomes more stable and gradually approaches a plateau. From roughly step 700 onward, the evaluation return mostly stays in the range of about 1600 to 1725, with relatively small variation between evaluation episodes. The best observed mean evaluation return is about 1724 (around step 900–960). At the final checkpoint (step 1400), the evaluation return mean is about 1648 with a standard deviation around 7.6, which suggests the policy is stable under stochastic demand noise.

6.2 Entropy Temperature (α) Trend

As in the figure 3 the entropy temperature α decreases smoothly over training. In the plot, α goes from about 0.20 down to around 0.144 by step 1400. This behavior is consistent with automatic entropy tuning in SAC. At the beginning, a higher α encourages more exploration

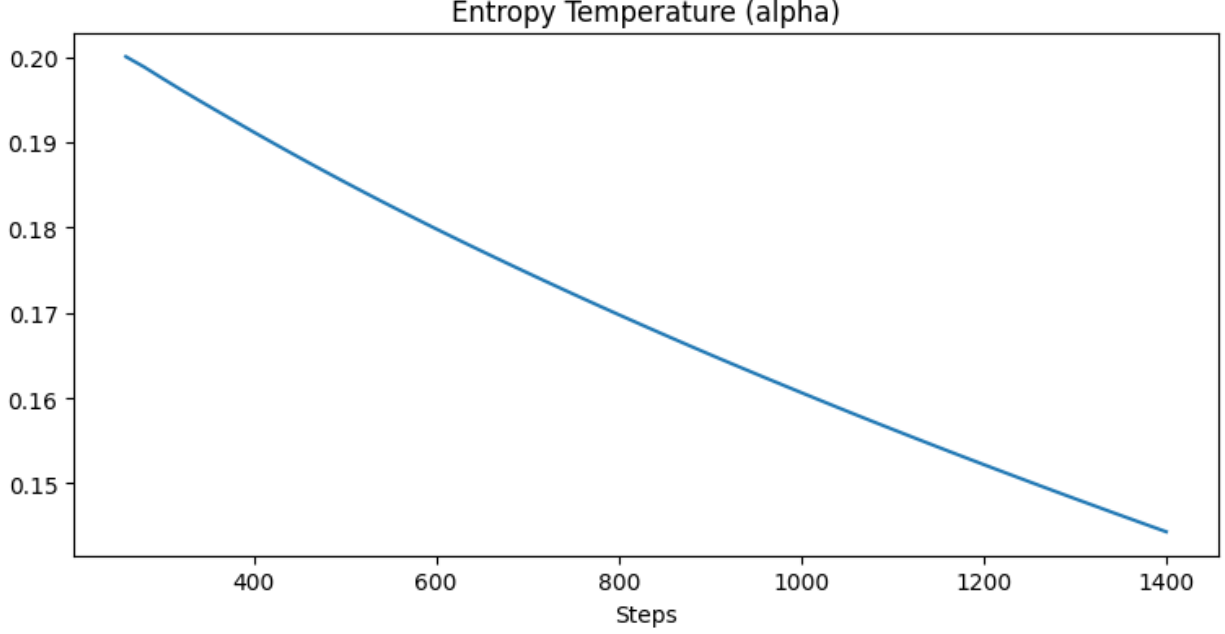


Figure 3: Entropy Temperature

by giving more weight to the entropy term in the SAC objective:

$$J(\pi) = \mathbb{E} \left[\sum_t (r_t + \alpha \mathcal{H}(\pi(\cdot | o_t))) \right]. \quad (30)$$

As training progresses, α becomes smaller, which means the policy becomes less exploratory and more focused on exploiting the learned profitable actions. This matches the observation that returns become more stable later in training.

6.3 Loss Curves and Training Stability

As in the figure 4 the loss curves also support the idea that training is stable overall. The critic (Q) loss starts relatively high (around 4–5 when updates first appear in the logs) and then drops to much smaller values (typically around 0.4–0.7 later). This suggests that the Q-functions learn to predict the soft Bellman backup more accurately over time.

The policy loss decreases steadily and becomes strongly negative later in training. In SAC, the policy is optimized using an objective of the form:

$$L_\pi = \mathbb{E}[\alpha \log \pi(a | o) - Q(o, a)]. \quad (31)$$

When the agent finds actions that have higher predicted Q-values, the second term dominates and the loss becomes more negative. Therefore, the decreasing policy loss is consistent with the agent learning actions that produce higher expected long-term rewards.

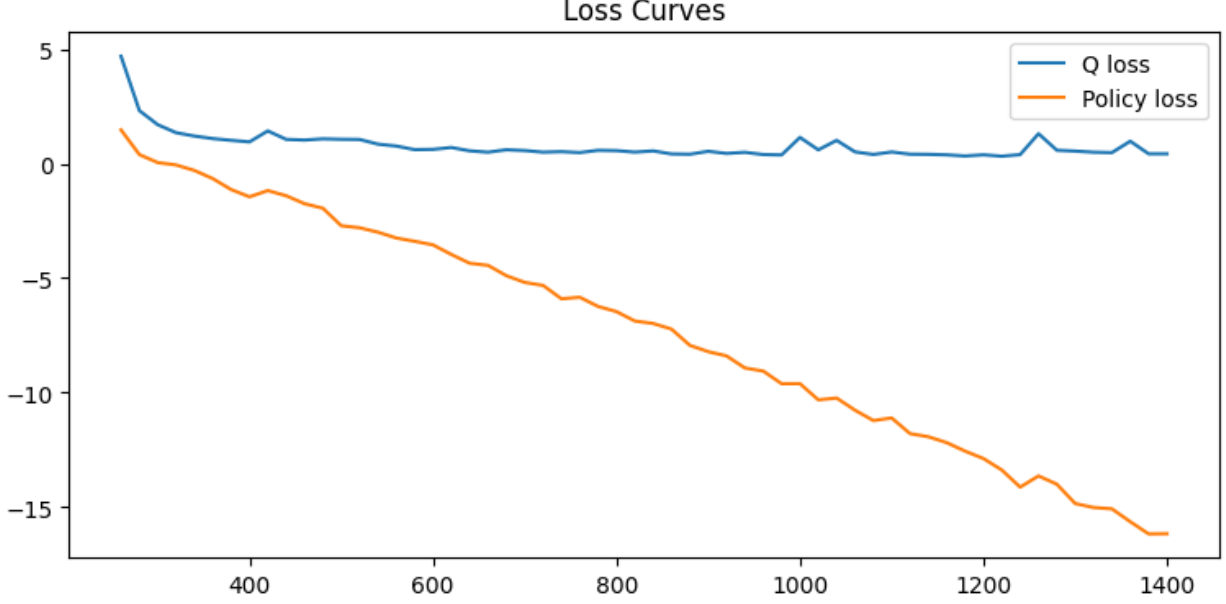


Figure 4: Loss Curves

6.4 Learned Pricing Behavior vs. Competitor

As in figure 5 after training, we ran a diagnostic rollout using the deterministic policy and plotted the agent’s price p_t against the competitor price p_t^c . The results show that, after a short initial transient at the beginning of the episode, both prices remain in a relatively narrow band.

In most steps, the agent’s price is slightly higher than the competitor’s price, typically around 10–13 for the agent while the competitor stays around 9.5–11.5. This pattern suggests that the learned strategy does not aggressively undercut the competitor. Instead, the agent appears to prefer a moderate markup while still staying close to the competitor, which can be reasonable because profit depends on both margin ($p_t - c$) and demand q_t .

6.5 Baseline Demand (α_t) and Realized Demand

The second diagnostic plot compares the baseline demand α_t and the realized demand q_t . The baseline demand shows clear seasonality and gradual growth, consistent with the simulator design:

$$\alpha_t = (\alpha_0 + g \log(1 + t/T)) (1 + s \sin(2\pi t/T + \phi)). \quad (32)$$

The realized demand is noisier, but it generally follows the same up-and-down seasonal pattern as α_t . Demand tends to be higher when α_t is high and lower when α_t drops, which is expected given the demand equation:

$$q_t = \max(0, \alpha_t - \beta p_t + \eta(p_t^c - p_t) + \epsilon_t). \quad (33)$$

Even though explicit time features were not included in the observation, the agent’s pricing behavior still remains stable across the seasonal changes, and the demand stays

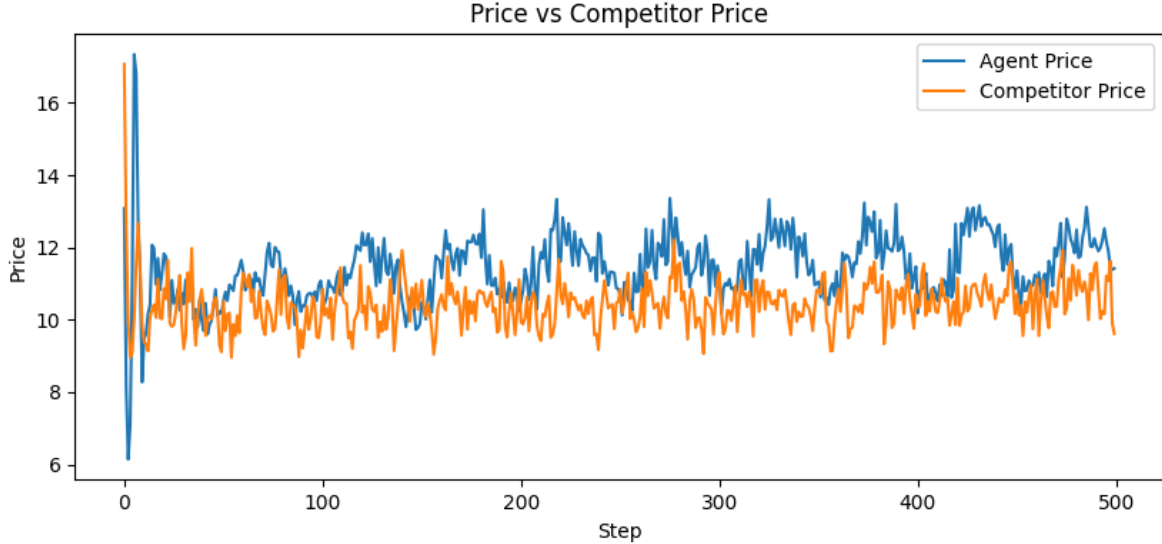


Figure 5: Price vs Competitor Price

mostly positive throughout the episode. This indicates that the history-based observation window provides enough information for the agent to react to changing market conditions in practice.

7 Discussion and Insights

This section discusses what the results imply about the learned policy and the environment. Overall, the experiments show that the SAC agent can learn a profitable and fairly stable pricing strategy in a competitive market, even when the market has noise and hidden demand patterns.

7.1 What the Agent Learned

From the rollout plots, the agent learned to set prices that stay close to the competitor but are usually slightly higher. This suggests the agent is not trying to start a price war by constantly undercutting. Instead, it seems to follow a **stable markup strategy**, where it keeps a reasonable margin $(p_t - c)$ while still maintaining enough demand. This makes sense because the reward is based on profit:

$$r_t = (p_t - c)q_t - \lambda|p_t - p_{t-1}|. \quad (34)$$

If the agent always chooses a very low price, it can increase demand but reduce margin. On the other hand, if it chooses a very high price, margin is larger but demand can drop. The final behavior looks like a compromise between these two effects. The learned prices mostly stay in a narrow range, which also indicates that the agent found a pricing region that works well across different demand fluctuations.

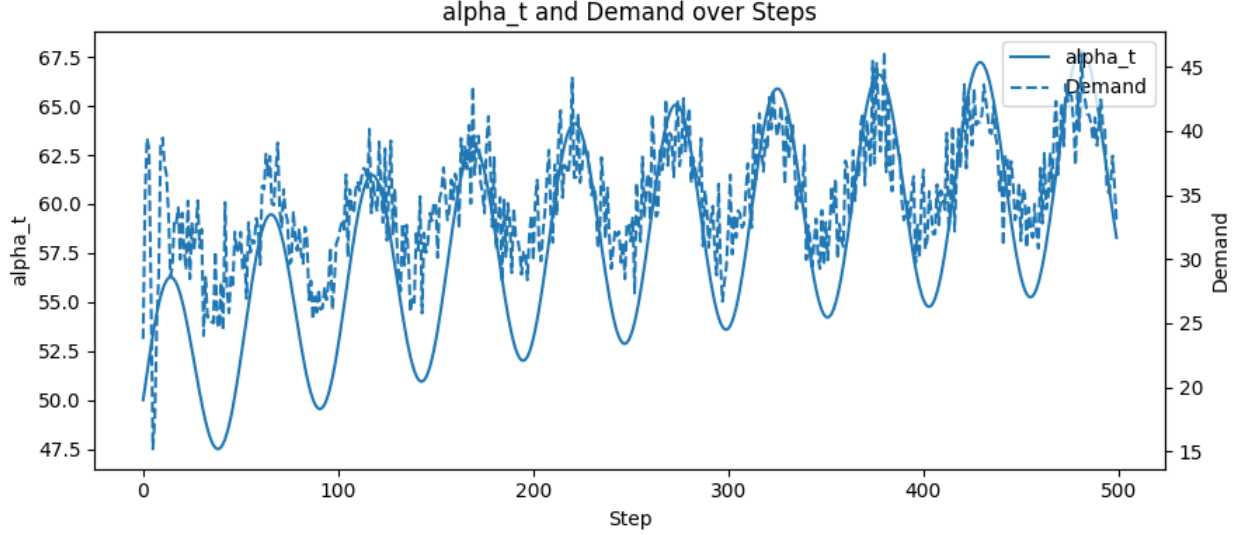


Figure 6: Baseline Demand (α_t) and Realized Demand

7.2 Effect of Partial Observability

A key point in this project is that the agent does not observe the true baseline demand α_t directly, and it also does not receive explicit time features. However, the results show that demand remains mostly positive and follows the seasonal pattern, while the agent’s pricing stays stable. This implies that the **history window** is enough for the agent to infer useful information about the market state. In other words, the agent is indirectly learning about hidden demand changes by using past demand and past prices in the observation:

$$\tilde{s}_t = [p_{t-k:t-1}, p_{t-k:t-1}^c, q_{t-k:t-1}]. \quad (35)$$

Even without knowing the exact time step or the seasonal phase, the agent can still react to changes in demand signals. This is important because in real markets firms often do not observe all drivers of demand, and they must rely on historical sales data to make decisions.

7.3 Stability vs. Profit Trade-off

The smoothness penalty term $\lambda|p_t - p_{t-1}|$ plays an important role in shaping the final behavior. The rollout plot shows that after the initial transient, the agent does not jump between very different prices. This suggests that the penalty successfully discourages unstable pricing. Without this term, the agent might try to overreact to noisy demand or small competitor changes, which could lead to large oscillations. With the penalty, the agent is encouraged to find prices that remain profitable while changing gradually.

In the evaluation curve, performance becomes stable in later training, and the returns remain high with low standard deviation. This stability matches the idea that the learned policy is not only profitable but also consistent across episodes.

7.4 Training Efficiency and Learnability

Another important insight is that the agent learns quite quickly once learning starts. At the beginning of training, evaluation returns are around -190 , but after updates begin, the return increases sharply and reaches above 1400 within a relatively small number of steps. This shows that the environment is learnable and that SAC can find a good strategy with limited interaction.

At the same time, the learning curve is not perfectly smooth, and there are some drops in performance at intermediate steps. This indicates that the problem is not trivial. The environment includes stochastic demand noise and a dynamic competitor, so the agent still needs time to stabilize its value estimates and policy. The gradual decrease of the entropy temperature α also supports this: early exploration is needed, but later the policy becomes more deterministic as it converges to a stable solution.

8 Limitations

This project uses a simplified simulator, so the results should be interpreted in that context. Demand is modeled as a linear function with additive noise, which may not capture real demand curves that can be nonlinear or have saturation. The competitor is also modeled with a stationary autoregressive rule. In real markets, competitors may learn and change strategy over time, which would create a more difficult multi-agent problem.

9 Future Work

There are several clear directions to extend this work. A natural next step is to model multiple competitors or a competitor that also learns, which would turn the problem into a true multi-agent learning setting. It would also be useful to use recurrent policies such as LSTMs, because they can handle partial observability in a more principled way than fixed history stacking. Another extension is to test different demand models, including nonlinear and data-driven demand estimators, and to evaluate the policy under distribution shifts where market conditions change.

10 Conclusion

In this project, we studied competitive dynamic pricing as a sequential decision-making problem and solved it using a deep reinforcement learning approach. We designed a simulated market environment where a firm sets a continuous price while competing with another dynamic seller, and where demand changes over time with stochastic noise and hidden seasonal patterns. The agent only observed a history window of past prices and demand, so the task was partially observable and closer to real-world pricing situations.

The results show that the SAC agent was able to learn an effective pricing strategy. During training, the evaluation return stayed low at the beginning, but once the learning updates started, performance improved rapidly. After this phase, the return reached a high

level and remained relatively stable, which indicates that the learned policy is not only profitable but also consistent across different evaluation episodes. The entropy temperature α decreased smoothly over time, meaning the agent explored more early in training and then became more deterministic as it converged.

The post-training rollout analysis also supports these findings. The learned prices stayed close to the competitor’s prices but were often slightly higher, which suggests the agent learned a stable markup strategy instead of aggressive undercutting. In addition, the demand trajectory followed the seasonal baseline demand pattern, and demand remained mostly positive. This implies that the agent could adapt to hidden market dynamics using only historical information, even without explicit time features.

Overall, the experiments demonstrate that reinforcement learning can successfully learn competitive pricing policies in a dynamic and uncertain environment. The final policy achieves strong long-term return while showing stable pricing behavior, which is an important requirement in practical pricing applications.